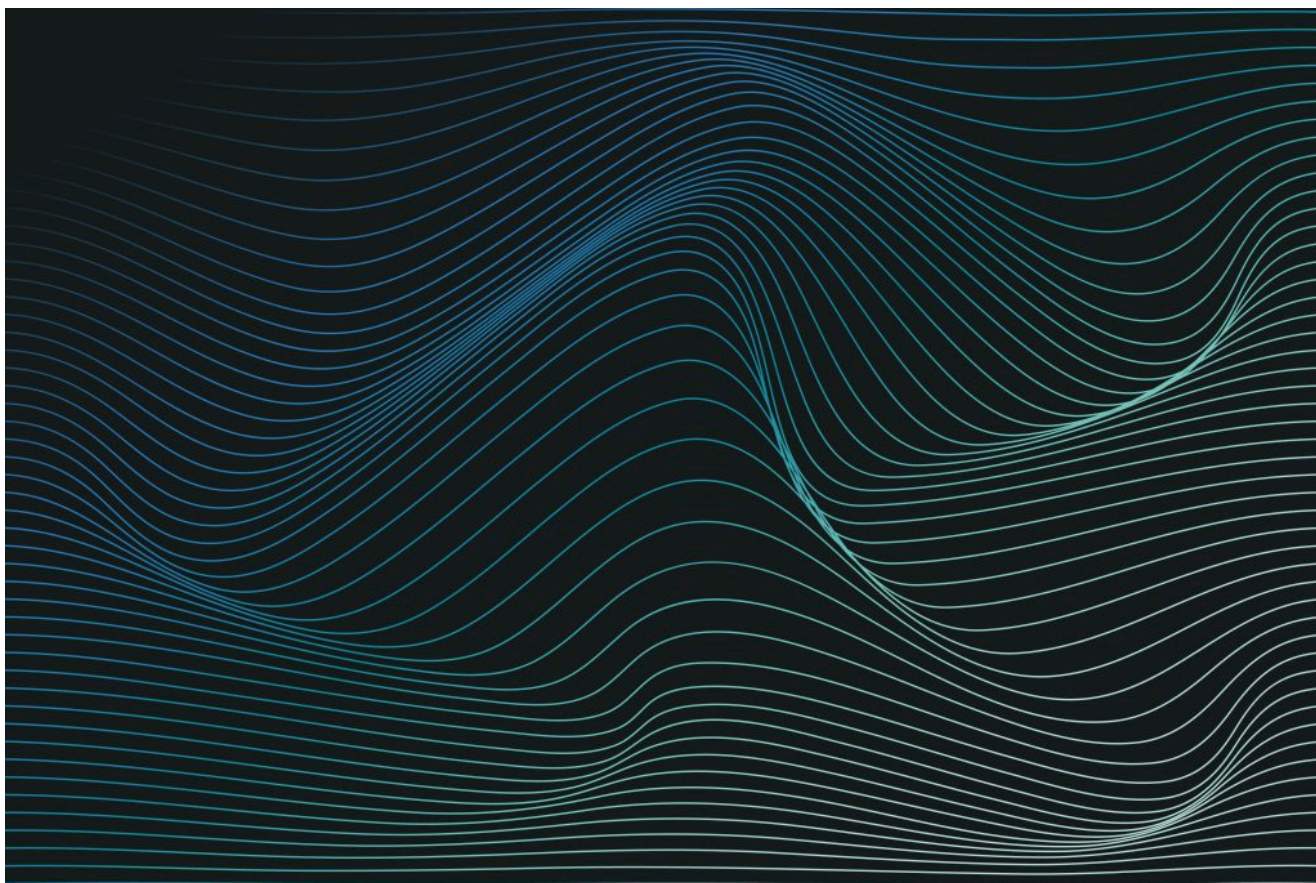


ИНФРАСТРУКТУРА ИСКУССТВЕННОГО ИНТЕЛЛЕКТА / ДАННЫЕ / БАЗЫ ДАННЫХ

Когда модель данных становится узким местом: уроки, которые можно извлечь из хранилища функций Medium

Как Medium перестроила свою модель данных хранилища функций на ScyllaDB для рекомендаций с низкой задержкой при 1 млн операций в секунду, а также сравнила с DynamoDB.

9 июня 2026 года, 10:40 [Синтия Данлоп](#)



Macude | Мариана Куэста для Unsplash+

[ScyllaDB](#) спонсировала этот пост.

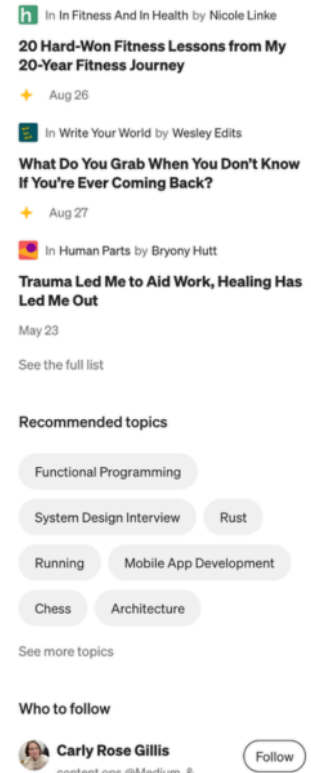
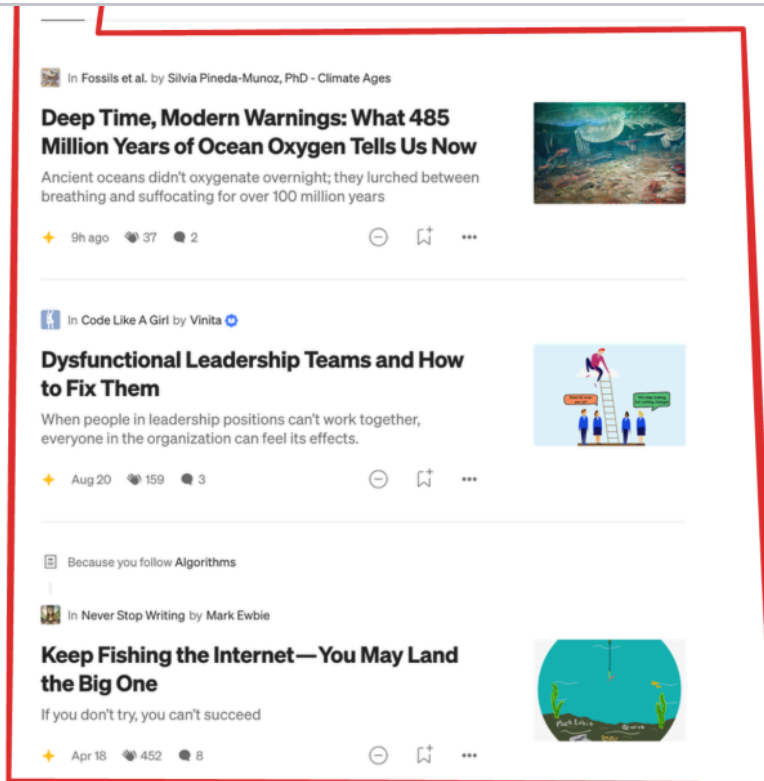
«Заставить читателей читать» — не такая уж простая задача для системы рекомендаций Medium. Чтобы предсказать, что с наибольшей вероятностью заинтересует конкретного читателя в любой момент времени, Medium непрерывно обрабатывает данные об активности пользователей (прочитанные истории, показанные рекомендации, подписки, лайки и т. д.).

Умные **модели и хорошая логика логического вывода** необходимы, но этого недостаточно. Данные должны храниться и извлекаться достаточно быстро, чтобы оставаться актуальными во время просмотра. Этим и занимается хранилище функций Medium. А правильная модель данных стала иметь большое значение, когда количество операций в секунду достигло 1 миллиона.

Андреас Содемон, главный инженер-программист компании Medium, недавно рассказал о том, как команда выявила проблему и что они сделали для ее решения. Если вы предпочитаете смотреть, а не читать, у вас есть два варианта: посмотреть короткую версию с **Monster Scale Summit** или расширенный **вебинар**

Хранилище функций и его роль в системе рекомендаций Medium

Хранилище функций объединяет все компоненты, обрабатывая пользовательскую активность и внутренние события и передавая их моделям машинного обучения, которые формируют рекомендации. Именно оно позволяет настраивать такие функции, как лента «Для вас», которая появляется у авторизованных пользователей.



ScyllaDB разработана для обеспечения предсказуемой производительности при масштабировании. Ее используют организации, которым нужна сверхнизкая задержка даже при миллионах операций в секунду и петабайтах данных. Наша уникальная архитектура использует возможности современной инфраструктуры, что позволяет сократить количество узлов, упростить администрирование и снизить затраты.

[Learn More](#) →

THE LATEST FROM SCYLLADB

Using Salting to Lower Latency for Large Blobs in ScyllaDB

25 June 2026

Riding the Raft to Strong Consistency in ScyllaDB

24 June 2026

Automate ScyllaDB X Cloud Clusters with Terraform

16 June 2026

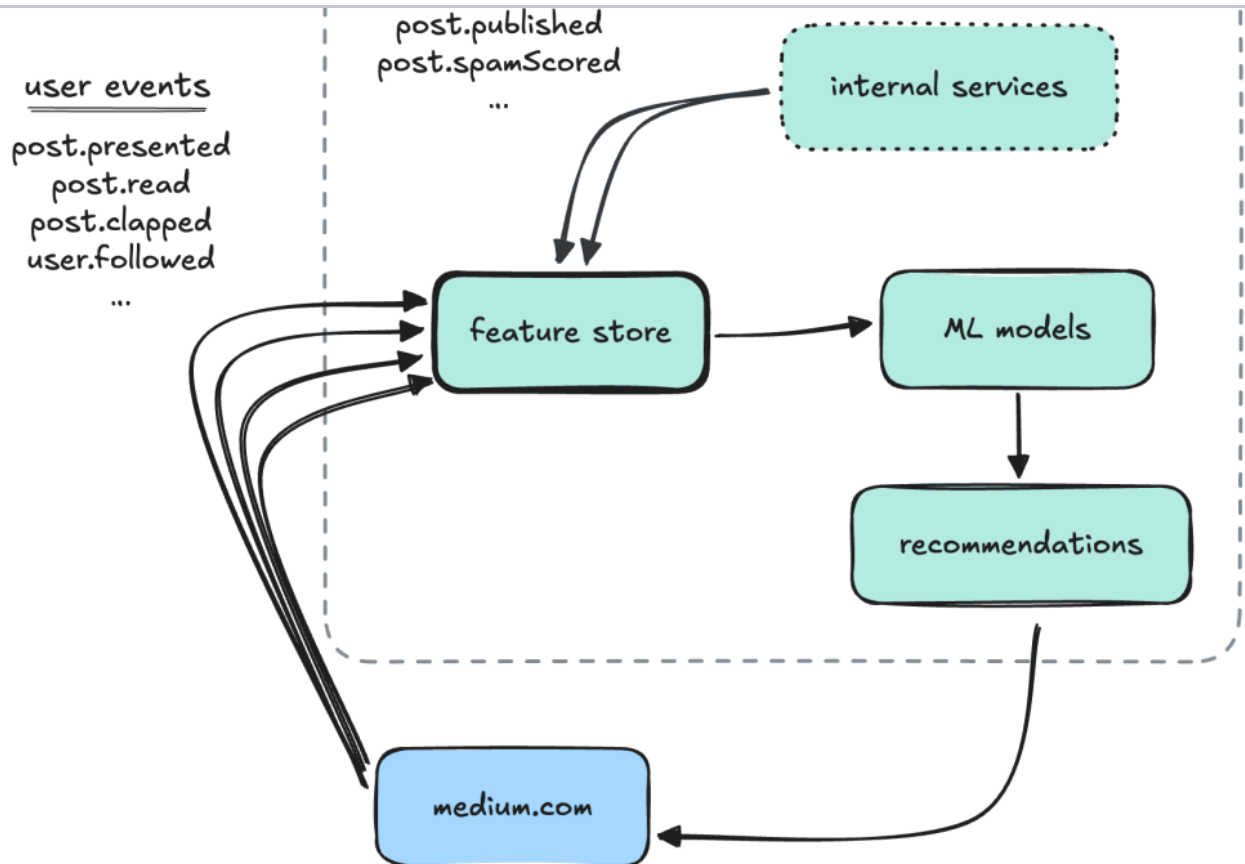
Submit

Each feature is a property of an entity, usually a **user or a story**. Some are simple and static, like whether a user holds a paid membership. Others capture interaction history: which stories a user has read, what content they've recently been shown, etc.

The following diagram shows a highly simplified view of the Medium feature store architecture:

TRENDING STORIES

1. The database storage problem is solved. Here's what comes next.
2. When your data model is the bottleneck: lessons from Medium's feature store
3. Why MotherDuck refuses to fork DuckDB
4. Why the need for humans won't disappear in the age of autonomous databases
5. For years, Apache Cassandra handed this work to your team — 6.0 takes it back

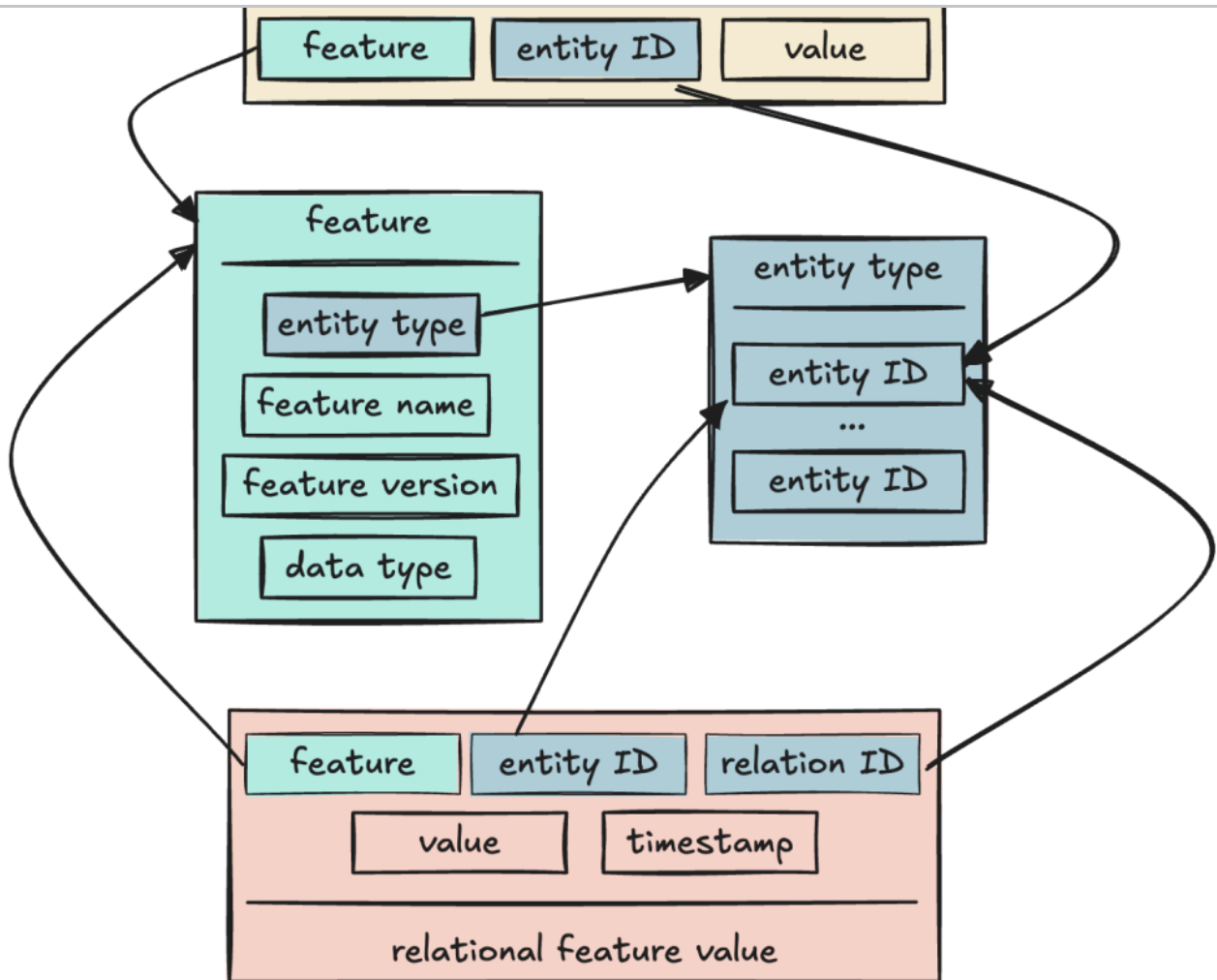


The problem with a relational features data model

When they built their feature store years ago, Medium used relational features for cross-entity relationships. Unlike regular features, a relational feature can have multiple values for a given entity ID. Each value is defined by a relation ID (the ID of the related entity) and a timestamp recording when the event occurred.

For example, a “story users have read” feature is attached to the story entity type. It relates to the user entity type, and its values indicate whether/when a given user has read that story.

Andréas shared the following schema diagram to explain the concept:

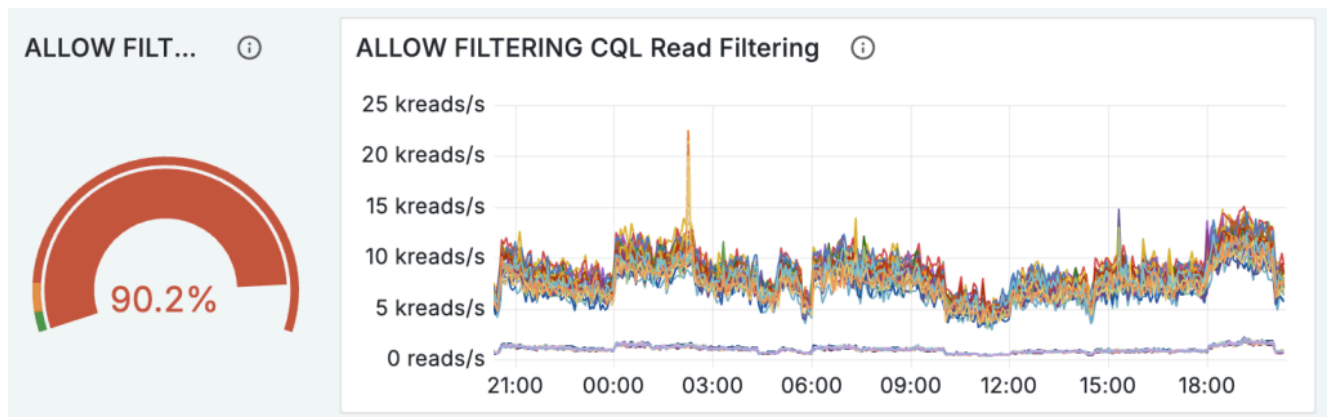


Features sit at the center, each attached to an entity type and defined by name, version, and data type. Non-relational features are simply a feature, an entity ID, and a value. Relational features add a relation ID mapping to another entity type, plus the value itself and a timestamp.

This approach proved suboptimal from a data modeling perspective. Since relational features link two entity types, the data ends up split between two tables: one for the entity IDs and one for the values. That means you can't get both in a single query. The first query retrieves only entity IDs (not their associated values) and relies on `ALLOW FILTERING`. A second query then runs for each entity ID to fetch its value. "If we have 1000 entity IDs for which we want to fetch values, then we have to run 1000 queries to fetch these values," Andréas said.

Overrelying on `ALLOW FILTERING` made things worse. "This is bad," Andréas said, referring to monitoring data showing that 90% of rows read via these queries were simply discarded. "This is just data that we don't need. `ALLOW_FILTERING` should be an escape hatch, not our design pattern."

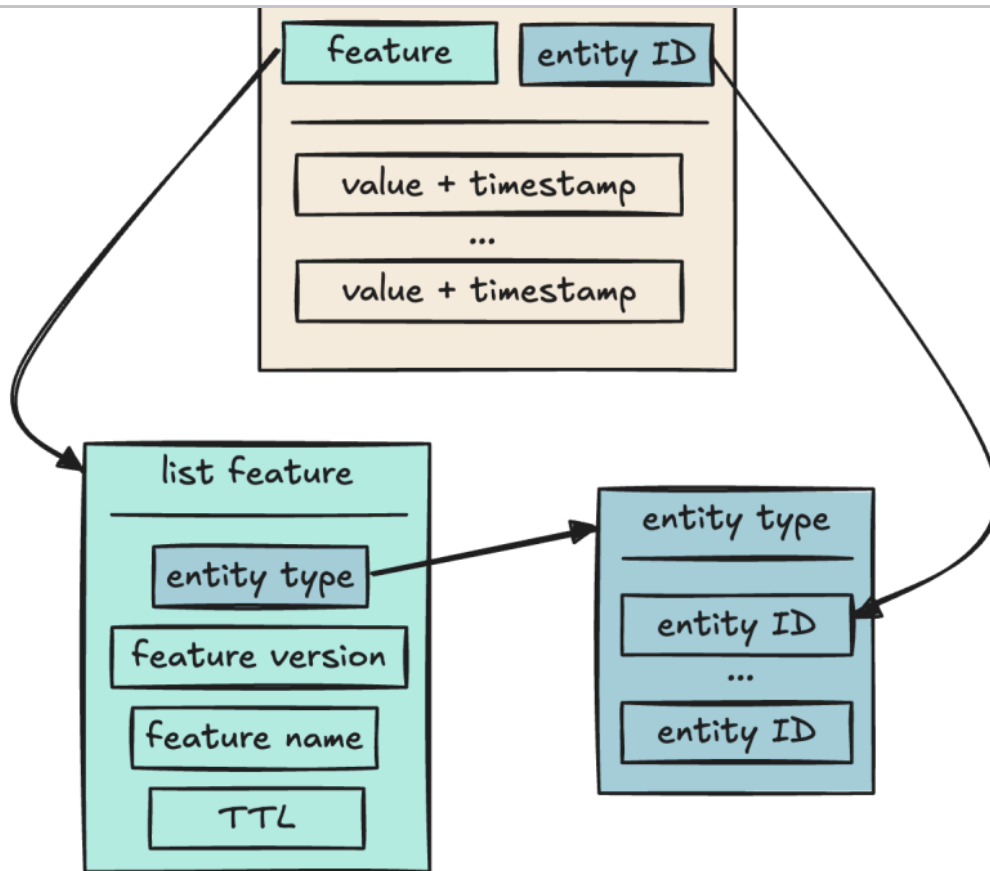
pattern.”



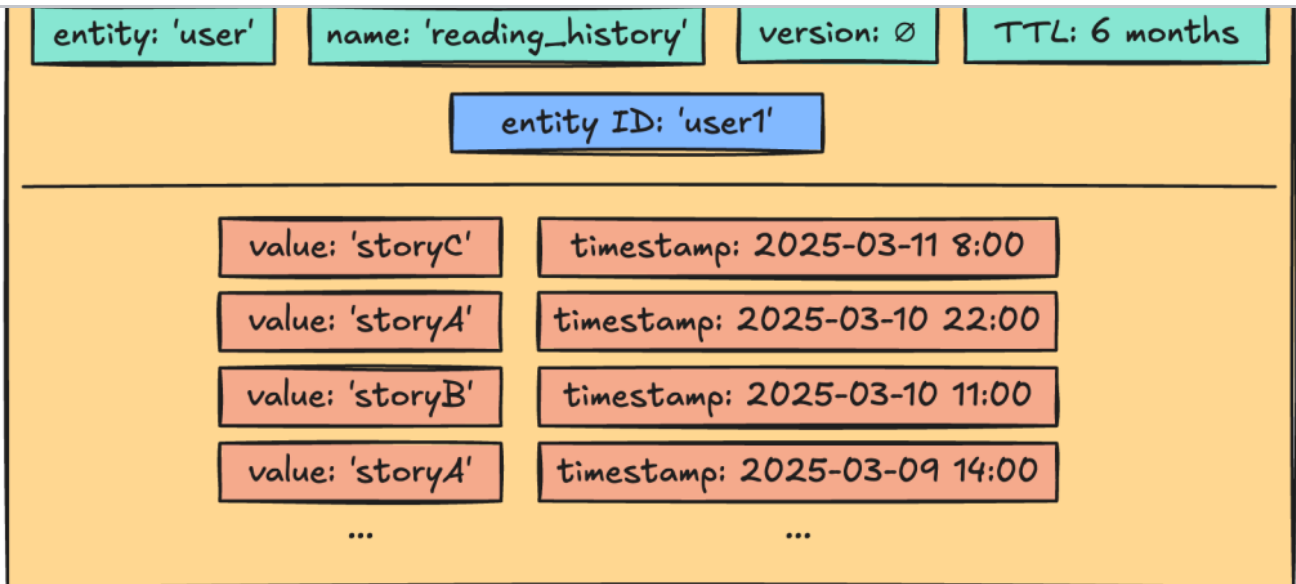
The list feature model

So they **reinvented their data model** and shifted to a list-based feature model. Instead of splitting data across two tables, everything for a given entity lives in one place and is retrieved in a single query.

Like other features, a list feature is defined by its entity type, name, and optional version. What's different is the value. While a non-relational feature has a single value, such as true or false, a list feature's value is a collection of items, each containing a value and a timestamp. Item values can be of any data type; the feature store doesn't enforce consistency within a list.



For example, consider a user's reading history. The entity is *user*, the feature name is *reading history*, the TTL is *6 months*. After that TTL is reached, the data is automatically dropped by the database (since older history isn't useful for recommendations). The list for a given user is a collection of story IDs and the timestamps at which they were read. The same story can appear multiple times, and multiple items can share the same timestamp.



A range of operations need to be supported. *Create List* and *Delete List* operations run at most a few times per day. *Remove List Items with Value*, which lets a reader scrub a specific story from their history so it stops influencing recommendations, runs at 1k-10k per second. *Add List Items* is higher still: every story read and every thumbnail shown to a user generates an event. *Get List Items* is the top, at 100k-1M operations per second.

Create List Feature	1/day
Delete List Feature	1/day
Add List Feature Version	10/day
Delete List Feature Version	10/day
Remove List Items with Value	1k-10k/second
Add List Items	10k-100k/second
Get List Items	100k-1M/second
Remove All List Items	?

are really the reasons why we need an efficient data store.”

“The *Add List Items*, and even more the *Get List Items* operations, are really the reasons why we need an efficient data store,” Andréas said.

Multiple items, one timestamp

Beyond raw efficiency, the new data model also had to support multiple items with the same timestamp. When Medium shows a user four story thumbnails simultaneously, all four presentation events share the same timestamp, but have distinct story IDs. If this isn’t handled correctly, primary key collisions occur.

The team’s solution was a single *list_items* table that stores everything.

The *list_items* Table

```
CREATE TABLE list_items (  
  feature_key TEXT,  
  entity_id TEXT,  
  item_key TEXT,  
  value BLOB,  
  PRIMARY KEY ((feature_key, entity_id), item_key)  
)  
WITH CLUSTERING ORDER BY (item_key DESC)  
AND DEFAULT_TIME_TO_LIVE = $defaultTTL;
```

Stores **all the list items**
for **all the list features**
managed by the feature store



The partition key combines *feature_key* and *entity_id*, keeping all items for a given list together. All of user 123’s reading history is stored in one partition, retrieved in one query. The clustering key concatenates each item’s timestamp with an MD5 hash of its value. The hash is what makes same-timestamp items with distinct values possible.

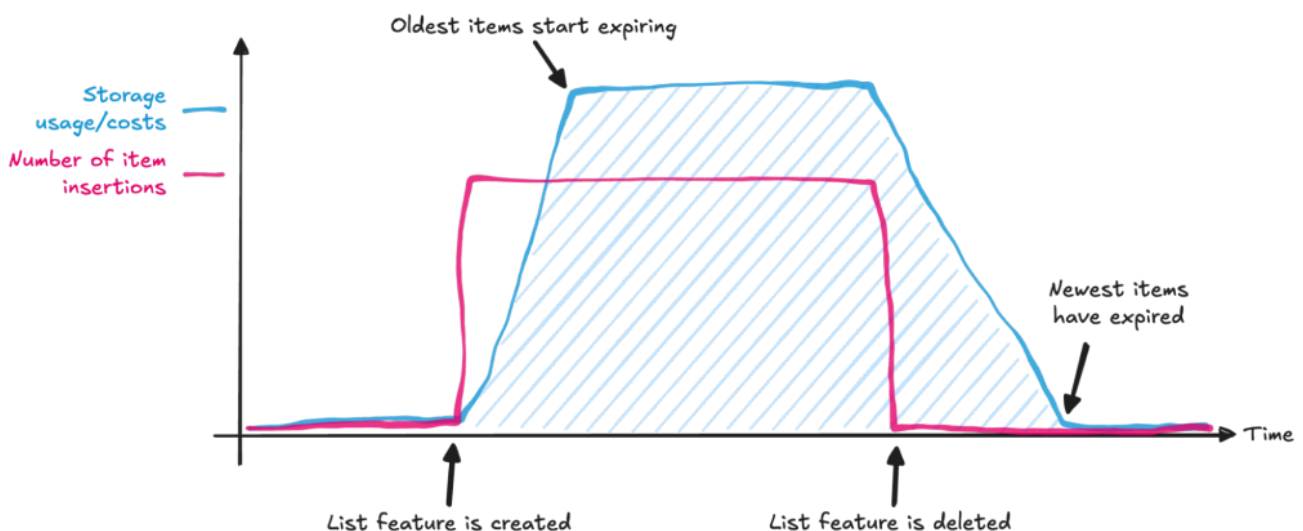
Using MD5 hashes for uniqueness raises its own set of questions, but in practice, the team hasn’t seen collisions. “The values that we are storing are

optimize for the typical read pattern (most recent N items) rather than leaving the application to sort afterward.

TTL to control storage costs

Storage cost is controlled entirely through ScyllaDB's native TTL, with no cleanup logic required. Every row expires automatically based on its own timestamp plus the feature's TTL duration. "We don't have anything to do regarding that," Andréas said. "Any row for which the TTL is expired will be considered deleted by ScyllaDB."

Storage plateaus for a steady write rate. When a feature is retired, its data drains away on its own. "That's super useful for controlling our storage and usage costs."

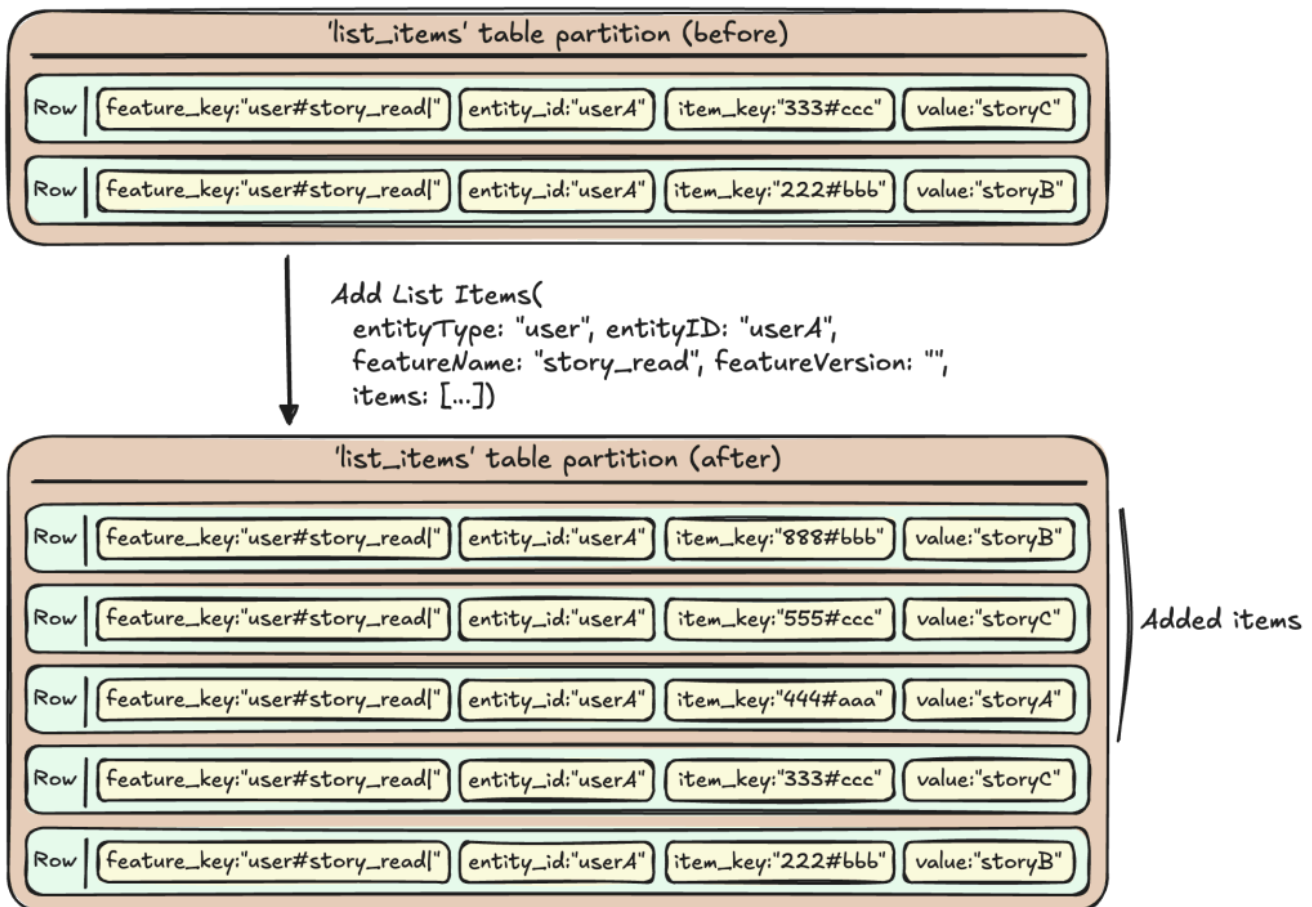


Implementing the list operations

Add List Items is a logged batch of INSERTs with atomicity guaranteed: all items land or none do. Each row carries its own TTL calculated from its timestamp, so older items expire sooner. Since items almost always carry a current timestamp, new entries append to the top of the partition, which is exactly where reads will look first.

```
BEGIN BATCH
-- for each $item in $items:
INSERT INTO list_items(feature_key, entity_id, item_key, value)
VALUES (
  '$EntityType#$featureName|$featureVersion',
  $entityID,
  ${buildItemKey(item)},
  ${item.Value},
)
USING TTL ${item.Timestamp + ttl - now};
APPLY BATCH; 1
```

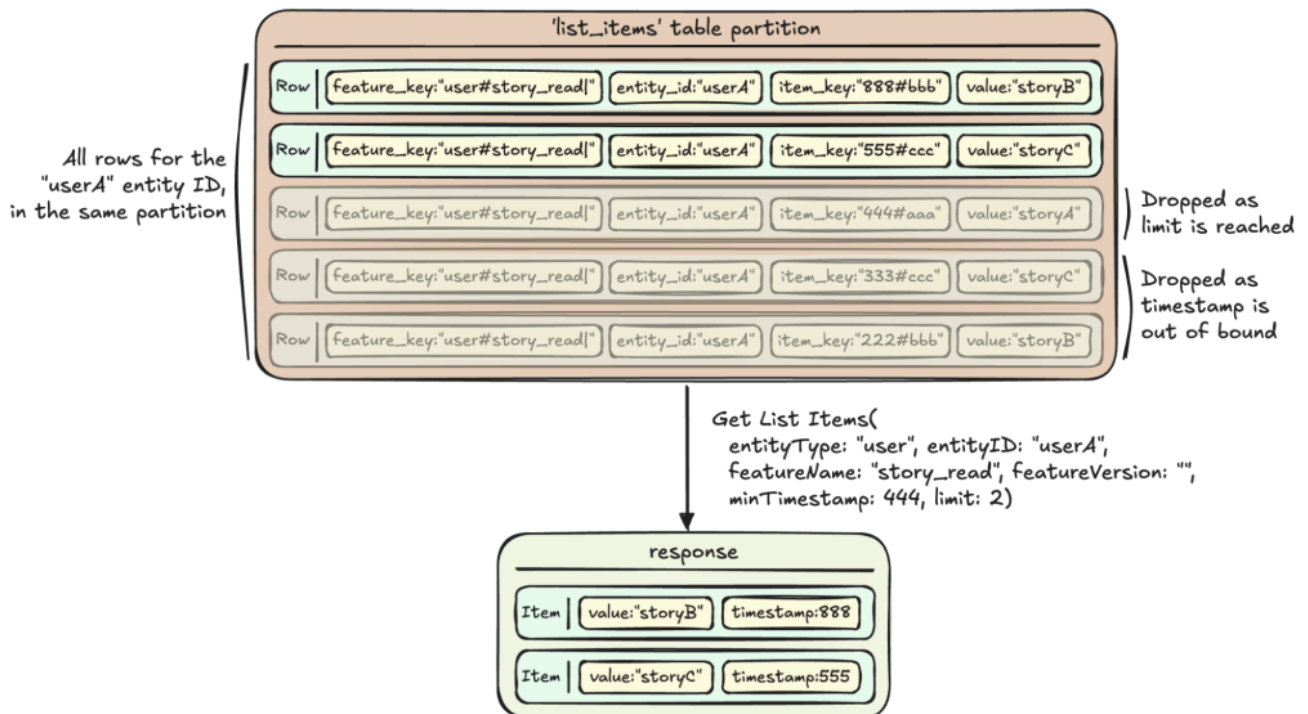
1 Logged batch (default) ensures that insertion of items is atomic



Get List Items runs as a single-partition SELECT with a minimum timestamp and a row limit. “We run the query on a single partition,” Andréas said. “That’s the maximum efficiency that we can have.” The clustering key handles filtering and ordering directly. Post-processing is not required.

```
SELECT value, item_key
FROM list_items
WHERE feature_key = '...' AND entity_id = $entityID 1
      AND item_key >= ${buildItemKey(minTimestamp)} 2
ORDER BY item_key DESC 3
LIMIT $limit;
```

- 1 Single partition for maximum efficiency
- 2 Discard old items
- 3 Efficient sorting via clustering key



Remove List Items with Value is the one operation that couldn't be reduced to a single query. Because value isn't part of the primary key, a direct filter isn't feasible.

```
SELECT item_key FROM list_items ❶  
WHERE feature_key = '...' AND entity_id = $entityID  
AND value = $value;  
  
-- for each batch of item_key values:  
DELETE FROM list_items ❷  
WHERE feature_key = '...' AND entity_id = $entityID  
AND item_key IN ($itemKeyBatch);
```

- ❶ Fetch keys of items to delete via list_items_by_value LSI
- ❷ Run batches of DELETE statements
- ❸ Only list operation that requires more than one requests to ScyllaDB 😞



A local secondary index built specifically for this case first finds the matching item keys, then a batch DELETE removes them by primary keys.

The list_items_by_value LSI

```
CREATE INDEX list_items_by_value  
ON list_items((feature_key, entity_id), value);
```

Used by the Remove List Items with Value operation

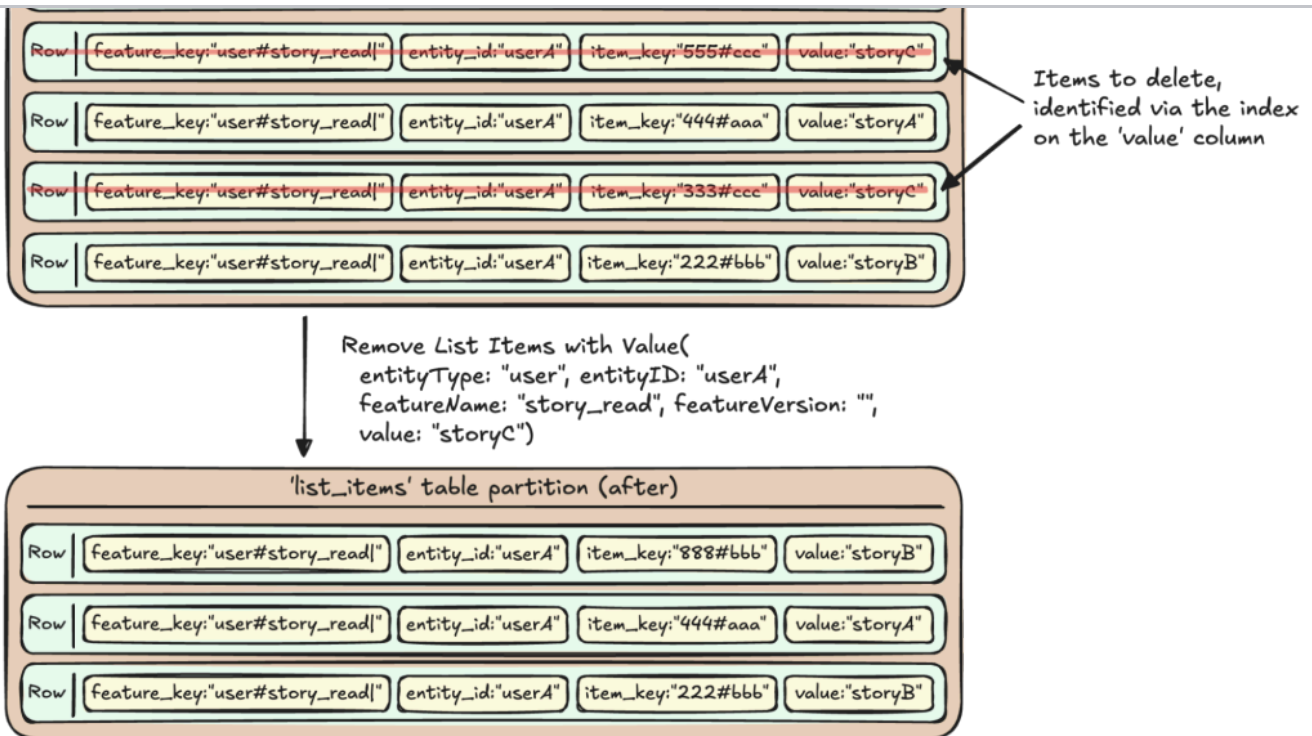
Local index ensures data is stored on same node as the base table

Faster than a scan as query is highly selective

Faster than a global index for our use cases



“Using an index is really faster than a scan because the query is highly selective,” Andréas explained. “We have very few items in a given list that have the same values compared to the total number of items in a list. And thanks to the current structure, using a local secondary index is faster than a global index.”

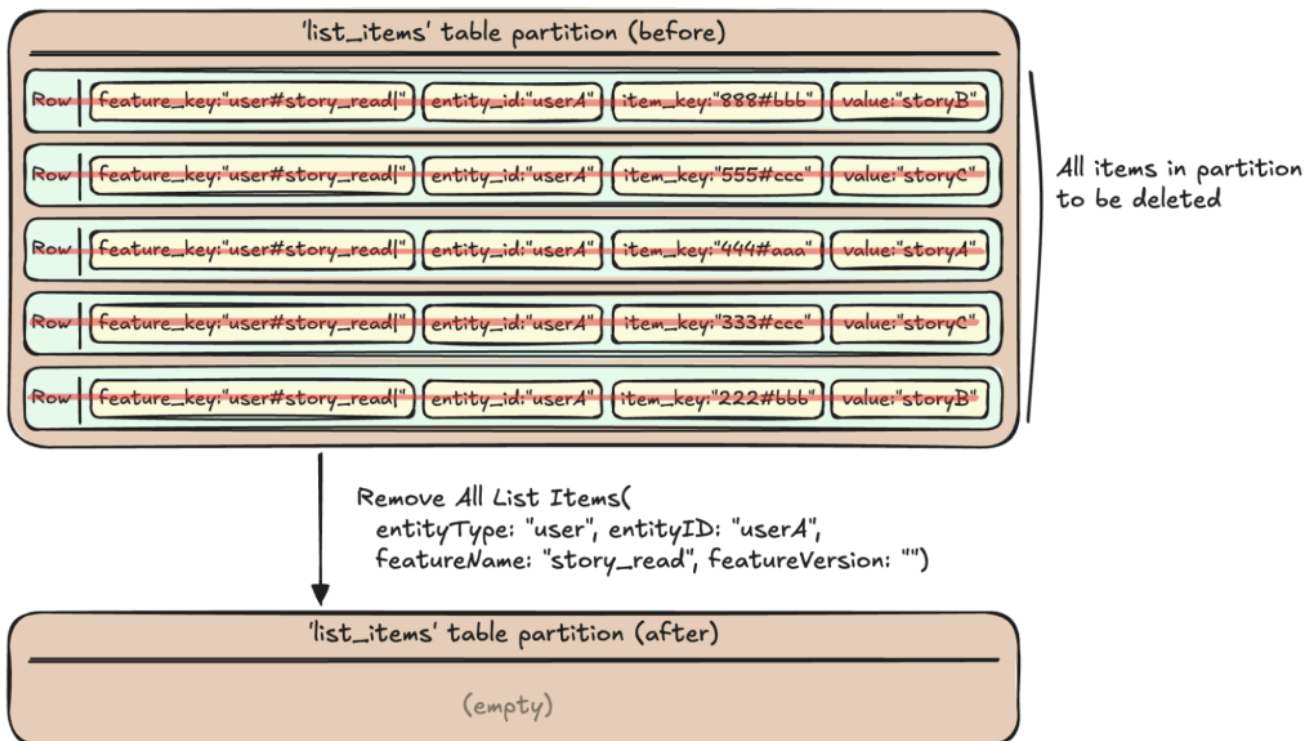


Andréas shared another example. Starting with the original table partition, the goal is to delete all items with the value “storyC.” Using the local secondary index, the system first identifies the two rows containing that value. It then issues two DELETE statements using the item keys from those rows, which removes them from the list. The final operation, removing all list items, is even more straightforward.

“We can just drop the partition,” Andréas said, “and ScyllaDB does its magic. It just deletes all the rows for that partition, which means that it deletes all the items for the given list. And bonus point: it’s atomic. It’s either completing successfully or not changing anything at all.

```
DELETE FROM list_items
WHERE feature_key = '...' AND entity_id = $entityID; 1
```

- 1 Deletion is atomic as we're deleting a whole partition

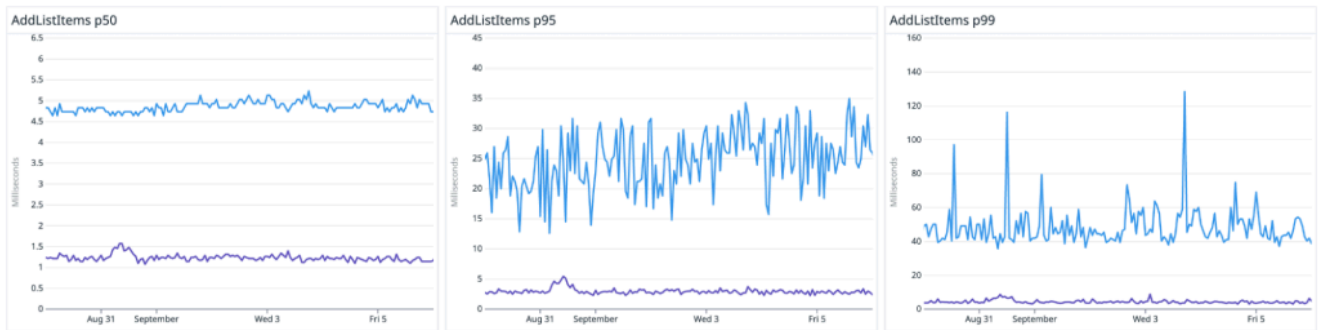


ScyllaDB vs. DynamoDB performance

Medium implemented the list operations on top of both ScyllaDB and DynamoDB. The main goal was to benchmark how both databases compared on their actual production data. “Conceptually they are very close,” Andréas noted, “but they have significant differences in how they operate.”

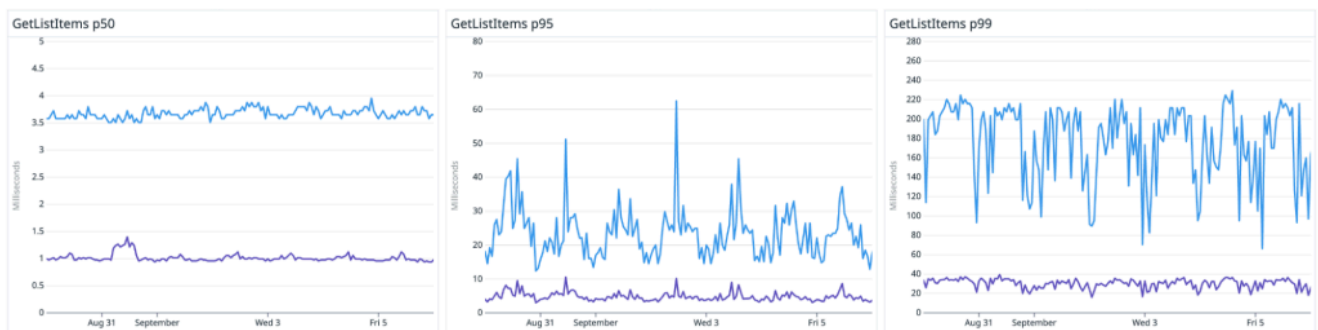
For *AddListItems*, P50 latencies were low with both databases: ScyllaDB came in at 1.5ms, DynamoDB under 5ms. “DynamoDB is extremely fast, not as fast as

ms P95, while DynamoDB ranged from 15-45 ms. ScyllaDB's P95s were steady single-digit milliseconds, while DynamoDB's ranged from 40- 120 ms.



AddListItem latencies: The blue line is DynamoDB; the purple line is ScyllaDB

It was a similar story for *GetListItem*s. At P50, ScyllaDB clocked in at 1 ms, DynamoDB at around 3.5 ms. At P95, ScyllaDB held around 5-6 ms while DynamoDB spiked from 30 – 60ms. And at P99, ScyllaDB remained at ~30ms while DynamoDB ranged from 70 ms all the way up to 220 ms.



GetListItem latencies: The top blue line is DynamoDB; the lower purple line is ScyllaDB

“ScyllaDB is very fast, with very predictable performance, and that’s super important for us.”

One caveat: DynamoDB was running without an extra caching layer. “We expect that could have a significant impact for DynamoDB because of the high cache hit rate that we are seeing on the list,” Andréas said. “But we don’t have the data yet, so we cannot compare them.” His verdict for now: “ScyllaDB is very fast, with very predictable performance, and that’s super important for us.”

y takeaways

because they didn't want to build on the shaky relational feature foundation.

Reflecting on the path to this point, Andréas left the audience with this parting advice:

“If you have a suboptimal data model, you will have queries that are slow, that will scale badly. And most likely, you won't be able to optimize that data model. You will have to define a new data model that will be better. So take time to think about your data model before you start the implementation, because once you have production data using your suboptimal data model, it's too late.”

TNS



Cynthia Dunlop has been writing about software development and testing for much longer than she cares to admit. She's currently senior director of content strategy at ScyllaDB.

[Read more from Cynthia Dunlop →](#)

SHARE THIS STORY



TRENDING STORIES

1. The database storage problem is solved. Here's what comes next.
2. When your data model is the bottleneck: lessons from Medium's feature store
3. Why MotherDuck refuses to fork DuckDB
4. Why the need for humans won't disappear in the age of autonomous databases
5. For years, Apache Cassandra handed this work to your team — 6.0 takes it back



ScyllaDB is engineered to deliver predictable performance at scale. It's adopted by organizations that need ultra-low latency, even over millions of ops/sec & PBs of data. Our unique architecture leverages the power of modern infrastructure – translating to fewer nodes, less admin & lower costs.

[Learn More →](#)

Using Salting to Lower Latency for Large Blobs in ScyllaDB

25 June 2026

Riding the Raft to Strong Consistency in ScyllaDB

24 June 2026

Automate ScyllaDB X Cloud Clusters with Terraform

16 June 2026

ScyllaDB Customer Experience Spotlight: Faisal Saeed

11 June 2026

ScyllaDB Operator 1.21 Release – with Oracle Kubernetes Engine (OKE) Support

10 June 2026

Dear cqlsh: Your dependencies were killing us (P.S. We rewrote you in Rust)

2 June 2026

TNS DAILY NEWSLETTER

**Receive a free roundup of the
most recent TNS articles in**

svo1975pnv1982@gmail.com

SUBSCRIBE

The New Stack does not sell your information or share it with unaffiliated third parties. By continuing, you agree to our [Terms of Use](#) and [Privacy Policy](#).

ARCHITECTURE

Cloud Native Ecosystem
Containers
Databases
Edge Computing
Infrastructure as Code
Linux
Microservices
Open Source
Networking
Storage

OPERATIONS

AI Operations
CI/CD
Cloud Services
DevOps
Kubernetes
Observability
Operations
Platform Engineering

THE NEW STACK

About / Contact
Sponsors
Advertise With Us
Contributions

ENGINEERING

AI
AI Engineering
API Management
Backend development
Data
Frontend Development
Large Language Models
Security
Software Development
WebAssembly

CHANNELS

Podcasts
Ebooks
Events
Webinars
Newsletter
TNS RSS Feeds



Community created roadmaps, articles, resources and journeys for developers to help you choose your path and grow in your career.

Frontend Developer Roadmap
Backend Developer Roadmap
Devops Roadmap

FOLLOW TNS



[Disclosures](#)

[Terms of Use](#)

[Advertising Terms & Conditions](#)

[Privacy Policy](#)

[Cookie Policy](#)